

OR Sentinel

Complete Implementation Guide

Real-time surgical safety AI on one HP ZGX Nano (GB10). HP Edge AI SJSUHack, deliverables due Friday, September 25, 2026 at 8pm.

This guide is self-contained. It explains every step in plain words, gives every command with what it does, and includes the complete code for every file. You should not need any other document to build, benchmark, demo and submit the project.

What you are building	One ZGX Nano listens to and watches 8 simulated operating rooms at once, tracks the sponge count by voice and by camera, and blocks closing if anything is missing. No patient data leaves the hospital; only anonymous protocol questions ever reach the cloud.
Who it is for	The circulating nurse, who owns the sponge and instrument count during surgery.
What makes it edge AI	Primary inference (speech understanding, sponge counting, reasoning) runs on the Nano. The cloud is used only when the local model is unsure, the question is not urgent, and all patient data has been removed.
How the code was checked	Every Python file in this guide compiles, and the full pipeline (simulator, rules, Critic, cloud router, tamper log, dashboard) was run end to end against a stand-in model server. It has NOT been run on a real GB10, so expect small fixes such as model names.

Contents

0. Read this first	4
How to use this guide	4
Team roles (for 4 people; merge roles if you have fewer)	4
Timeline (Thursday evening to Friday 8pm)	4
Plain-words glossary	4
1. How the system works	6
Why each design choice scores points	6
Final repository layout	6
2. Connect to the Nano and set it up	8
2.1 Connect from your laptop (everyone)	8
2.2 Install and configure ZRT (Person B, once)	8
2.3 Hugging Face account and token (Person A)	8
2.4 Useful commands you will use all week	8
3. Create the project and Python environment	10
4. Datasets	11
4.1 Resources and links	11
4.2 Sponge photo dataset (Person A, about 90 minutes)	11
4.3 Acted operating room audio (Person C, about 60 minutes)	11
4.4 Test scenarios list (Person D, 20 minutes)	12
5. Label the photos automatically	13
6. Fine-tune the vision model with HP's repo	15
6.1 Get HP's repo and test it	15
6.2 Find where it loads the medical data	15
6.3 Start training in tmux	16
6.4 Check training is healthy	16
6.5 When it finishes	16
7. Serve both models with ZRT	17
7.1 Download the models	17
7.2 start.sh and stop.sh	17
7.3 Check the services are up	18
8. Turn the audio into timestamped text	20
9. Build the app	21
9.1 agents.py - the four AI agents	21
9.2 fusion.py - the safety rules	22

9.3 ledger.py - tamper-proof log	23
9.4 cloud_router.py - the local vs cloud decision	24
9.5 simulate.py - 8 rooms at once	26
10. Dashboard	29
11. Benchmarks and metrics	31
11.1 Vision accuracy: base vs fine-tuned	31
11.2 Latency at 1, 4 and 8 rooms	31
11.3 Cloud escalation rate	32
11.4 Mismatches caught	33
11.5 Charts for slides	33
11.6 Metrics table for README and slides (fill in your numbers)	33
12. Chaos mode and the demo	35
12.1 Cut the cloud (live, safe)	35
12.2 Kill the vision model (live)	35
12.3 Tamper with the record (live)	35
12.4 Five-minute demo script	35
12.5 Answers to the questions judges will ask	35
13. Package for replicability	37
13.1 .gitignore and Dockerfile	37
13.2 README.md template (copy, then fill in the brackets)	37
13.3 Push to a public GitHub repo	38
14. Record, submit and post	39
14.1 Two-minute video (YouTube, Public)	39
14.2 Final checklist	39
15. Troubleshooting	40
15.1 What might break and the fallback	40
15.2 Error messages	40

0. Read this first

How to use this guide

- Follow the parts in order. Each part says who should do it, how long it takes, and what you should see when it worked.
- Every command goes into the terminal ON THE NANO (the VS Code ZTK terminal) unless it says **on your laptop**.
- Grey boxes are exact commands or file contents. Copy them exactly. Text after # is an explanation and is ignored by the computer.
- Yellow boxes warn about things that commonly go wrong. Green boxes are tips.
- Replace anything in angle brackets, such as <your-hf-username>, with your own value.

Team roles (for 4 people; merge roles if you have fewer)

Person	Owns	Parts of this guide
A - Data and training	Sponge photos, labels, fine-tuning, vision accuracy benchmark	4, 5, 6, 10.1
B - AI backend	ZRT serving, agents, fusion rules, cloud router, simulator	2, 7, 9
C - Frontend and audio	Acted audio clips, transcription, dashboard	4.3, 8, 10 (dashboard), 12
D - Evidence and delivery	Repo, benchmarks, charts, README, video, submission	3, 11, 13, 14

Timeline (Thursday evening to Friday 8pm)

When	Goal	Done when
Thu 6-9pm	Everyone connected; ZRT and Python set up; photos shot and labeled; audio recorded	Part 2 checks pass; data/train.jsonl exists
Thu 9-11pm	Fine-tuning started in tmux (runs overnight); 27B model served; agents tested	train.log shows loss going down
Fri 8am-12pm	Fine-tuned model served; simulator and dashboard working with 1 room, then 8	Room 6 turns red on the dashboard
Fri 12-4pm	All benchmarks, charts, chaos demo, README	bench/results has scale.png and vision.png
Fri 4-7pm	Record 2-min video and backup demo; upload; post socials; SUBMIT	Everything in Part 14 checklist ticked
Fri 7-8pm	Buffer only. Do not start anything new.	

WATCH OUT: If the Nano restarts, Tailscale turns off and nobody can connect remotely. Someone must go to the lab, use the keyboard and monitor plugged into the Nano, and type `sudo tailscale up`. Never run `sudo reboot` and walk away.

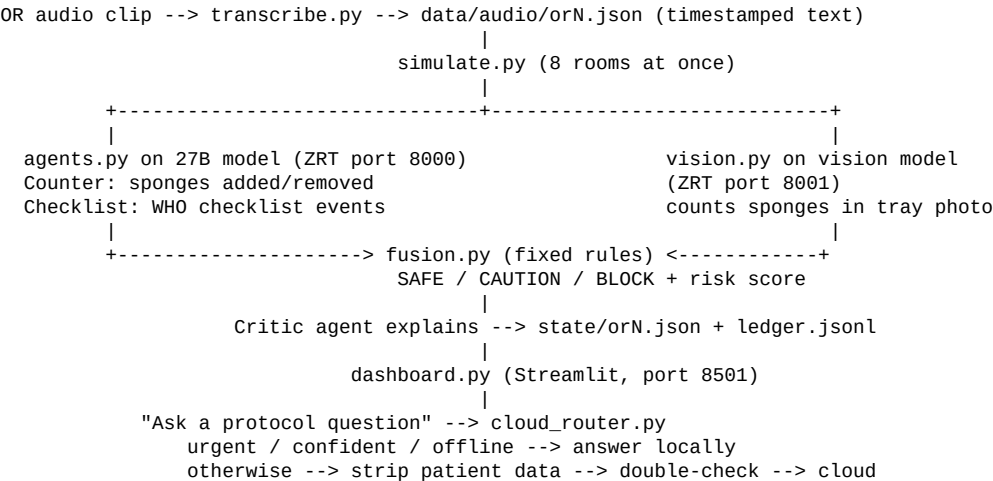
Plain-words glossary

Term	What it means here
ZGX Nano / GB10	HP's small AI computer with an NVIDIA GB10 chip and 128 GB of memory shared by CPU and GPU. It runs all the AI.
SSH	A secure way to type commands on another computer from your laptop.
Tailscale	A private network so your laptop can reach the Nano.

Term	What it means here
ZTK	HP's VS Code extension that connects VS Code to the Nano over SSH.
ZRT (Z Runtime)	HP's tool that downloads AI models from Hugging Face and runs them as a web service. It is built on vLLM.
vLLM	Fast software for running AI models. It handles many requests at once, which is why one Nano can serve 8 rooms.
Model	The AI itself. We use two: a 27B text model for reasoning and a 7B vision model for counting sponges in photos.
Fine-tuning	Extra training on your own examples so the model gets better at your specific task.
NVFP4	A compressed number format that makes models smaller and faster on the GB10.
Port	A numbered door on a computer. The 27B model listens on 8000, the vision model on 8001, the dashboard on 8501.
API / endpoint	The web address your code sends requests to, like <code>http://localhost:8000/v1</code> .
OpenAI-compatible	ZRT accepts the same request format as OpenAI, so the same Python code can talk to the local model or a cloud model.
JSON	Structured text like <code>{"added": 5}</code> . We make the AI answer in JSON so code can read it.
tmux	Keeps a terminal program running even if your laptop disconnects.
venv	A private Python install for this project so packages don't clash.
Hugging Face / token	The website that hosts models; a token is your password for scripts.
PHI / HIPAA	Protected health information (names, dates of birth, record numbers) and the US law protecting it.
Presidio	Microsoft's free tool that finds and hides personal details in text.
p50 / p95 latency	p50 = typical response time. p95 = the time 95% of responses beat; it shows the slow cases.
Streamlit	A Python library that turns a script into a web dashboard.
Docker	Packages an app with everything it needs so others can run it the same way.

1. How the system works

Audio from each operating room becomes text. The 27B model reads each line and keeps the spoken sponge count and checklist. When a sponge comes out, the vision model counts sponges on the tray photo. Fixed rules compare what was heard with what was seen and decide SAFE, CAUTION or BLOCK. The Critic agent explains the decision in one sentence, and the dashboard shows all 8 rooms.



Why each design choice scores points

Choice	Why judges care
Rules decide the block, AI only reads and explains	Answers 'what if the AI is wrong?' Safety never depends on a model's mood.
Audio AND camera	Audio alone would miss the hidden-problem room. Two independent sensors disagreeing is real systems design.
8 rooms on one Nano	Answers 'why not a cheap mini PC?' and shows vLLM batching on the GB10.
Explicit cloud rule, patient data stripped and double-checked	The challenge's core requirement: an explicit, defensible, measurable escalation decision.
Fine-tuning on the Nano with HP's repo	Uses HP's full train, push, pull, serve workflow. Training data never leaves the hospital either.
Tamper-proof ledger and chaos mode	Shows production thinking: audit trails and graceful failure.

Final repository layout

or-sentinel/	
start.sh	starts both models with ZRT (Part 7)
stop.sh	stops them
requirements.txt	Python packages (Part 3)
Dockerfile	replicable container (Part 14)
README.md	project write-up (Part 14)
.gitignore	keeps secrets and live files out of GitHub
app/	
config.py	all settings
make_labels.py	photo filenames -> training labels
test_vision.py	quick vision test
transcribe.py	audio -> timestamped text
agents.py	Counter, Checklist, Critic, Scribe
vision.py	sponge counting via the vision model
fusion.py	the safety rules
ledger.py	tamper-proof log
cloud_router.py	local vs cloud decision
simulate.py	replays 8 rooms at once
dashboard.py	command center

```
bench/
  eval_vision.py      base vs fine-tuned accuracy
  scale.sh, collect.py latency at 1, 4, 8 rooms
  escalation_eval.py  how often the cloud is used
  questions.txt       20 test questions for the router
  charts.py           slide charts
  results/            outputs (created automatically)
data/
  photos/             your tray photos (Part 4)
  audio/              your .wav clips and their .json transcripts
  train.jsonl, test.jsonl created by make_labels.py
state/                live status files (created automatically, not committed)
```

2. Connect to the Nano and set it up

Who: everyone connects; Person B does the one-time setup. **Time:** 30 to 45 minutes.

2.1 Connect from your laptop (everyone)

1. On your laptop, join the **edroam** Wi-Fi with your .edu email.
2. Install **Tailscale** from tailscale.com. The admins log each participant into the shared network (account zgxdemo@gmail.com). For the password, contact Sandor at SJSU or Wade at HP (wade@hp.com), as listed in HP's hackathon PDF.
3. Install **VS Code**, open the Extensions panel, search **ZGX Toolkit (ZTK)** and install it.
4. Click the ZTK icon, then **Add Device**, then **Discover Devices**. Pick your team's device. If it isn't listed, type the Tailscale IP the admins gave your team.
5. Keep the default device name (for example hp7). Username is the same (hp7). Password is edgehack. Click **Add Device**.
6. Select **Linux**, follow the prompts to test the connection, install Z Runtime (ZRT) if offered, then click **Connect**.
7. Open a terminal in that VS Code window (Terminal menu, New Terminal). The prompt should show your hp username. Every command from now on goes here.

WATCH OUT: Only one team per device. Never SSH into another team's Nano. Several teammates can connect at once, but coordinate heavy GPU jobs (training vs serving) so you don't slow each other down.

2.2 Install and configure ZRT (Person B, once)

```
sudo snap install --classic zrt          # install HP's model runner (skip if ZTK already did)
sudo usermod -aG docker zrt             # let ZRT use Docker
groups zrt                               # check: output must include the word docker
zrt config set proxy.auth.type none && zrt config set proxy.tls.enabled false
                                          # turn off ZRT's proxy login and HTTPS for local use
zrt status                               # expected: ZRT reports it is running
zrt models                               # lists downloaded models (empty is fine for now)
zrt --help                               # shows all ZRT commands
```

WATCH OUT: If groups zrt does not show docker, the Nano may need a restart for it to take effect. Only restart with someone physically in the lab to run `sudo tailscale up` afterwards.

2.3 Hugging Face account and token (Person A)

1. Create a free account at huggingface.co.
2. Profile picture, then **Settings**, then **Access Tokens**, then **Create new token**. Choose **Write** permission. Copy the token (starts with hf_).
3. Click your profile, then **New Model**. Name it or-sentinel-vision. Choose **Public** (judges may want to see it).
4. Save the token and repo name on the Nano:

```
echo 'export HF_TOKEN="hf_PASTE_YOUR_TOKEN"' >> ~/.bashrc          # save token for every new terminal
echo 'export HF_REPO_ID="<your-hf-username>/or-sentinel-vision"' >> ~/.bashrc
source ~/.bashrc                                                    # load the new settings now
echo $HF_TOKEN                                                       # check: prints your token
echo $HF_REPO_ID                                                     # check: prints your repo name
```

WATCH OUT: Never put the token in a file you commit to GitHub. The .gitignore in Part 14 excludes .env files, but the safest place is ~/.bashrc as above.

2.4 Useful commands you will use all week

Command	What it does
<code>nvidia-smi</code>	Shows whether the GPU is busy and which programs use it.
<code>free -h</code>	Shows how much of the 128 GB memory is used. On the GB10 this is the most reliable memory number.
<code>tmux ls</code>	Lists running tmux sessions (llm, vlm, train).
<code>tmux attach -t llm</code>	Opens the llm session to see its output. Press Ctrl+B then D to leave it running.
<code>tmux kill-session -t vlm</code>	Stops the program in the vlm session.
<code>tail -f file.log</code>	Shows new lines of a log as they appear. Ctrl+C stops watching (not the program).
<code>ls, cd folder, pwd</code>	List files, change folder, show current folder.
<code>curl localhost:8000/v1/models</code>	Asks the 27B model service which model it is serving.

3. Create the project and Python environment

Who: Person D. **Time:** 15 minutes. Afterwards, push to GitHub so everyone works from the same code.

```
mkdir -p ~/or-sentinel/app ~/or-sentinel/bench ~/or-sentinel/data/photos \
~/or-sentinel/data/audio ~/or-sentinel/state
cd ~/or-sentinel
python3 -m venv .venv          # private Python install for this project
source .venv/bin/activate      # switch into it; your prompt now starts with (.venv)
```

WATCH OUT: Run `source ~/or-sentinel/.venv/bin/activate` in EVERY new terminal before running Python. If you see `ModuleNotFoundError`, you forgot this.

Create `requirements.txt` with exactly this content (in VS Code: New File, paste, save into `~/or-sentinel`):

File: requirements.txt

```
openai>=1.40
streamlit>=1.37
matplotlib
pillow
faster-whisper
presidio-analyzer
presidio-anonymizer
spacy

pip install --upgrade pip
pip install -r requirements.txt      # installs everything the app needs (a few minutes)
python -m spacy download en_core_web_lg  # language model Presidio uses to spot names (~500 MB)
pip list | grep -i -E "openai|streamlit|presidio|faster"  # check they installed
```

Now create every file from Part 9 onward in the `app/` and `bench/` folders as you reach it. The fastest way: in VS Code's file explorer (connected to the Nano), right-click the folder, New File, paste the grey box, save.

4. Datasets

You make your own data. No public dataset shows 'gauze sponges on an operating room tray with counts,' and making your own takes about two hours in total. It is also a great story for judges: show a photo of your mock operating room.

4.1 Resources and links

What	Link	Used for
HP fine-tuning example	https://github.com/jrgosalvez/Med_VLM_Fine-Tune_vLLM	Training pipeline you adapt in Part 6
Base vision model	https://huggingface.co/Qwen/Qwen2.5-VL-7B-Instruct	Model you fine-tune
27B reasoning model	Inferact/Qwen3.8-27B-NVFP4 (name from HP's PDF)	All four agents
faster-whisper	https://github.com/SYSTRAN/faster-whisper	Speech to text
Microsoft Presidio	https://github.com/microsoft/presidio	Removing patient data
Streamlit docs	https://docs.streamlit.io	Dashboard
Freesound	https://freesound.org	Hospital monitor beeps for background audio (choose CC0 clips)
Roboflow Universe (optional)	https://universe.roboflow.com	Extra surgical images if you want more (check each license)
WHO Surgical Safety Checklist	Search that name on who.int	Official checklist items your Checklist agent tracks

WATCH OUT: These links were written without web access. Open each one and confirm it works and that its license allows your use before you rely on it.

4.2 Sponge photo dataset (Person A, about 90 minutes)

Shopping list: 2 packs of 4x4 inch gauze pads (about 50 pads), one metal or plastic tray, red food coloring, one dark and one white cloth for backgrounds.

Shooting protocol (phone camera is fine):

1. For each count from 0 to 15, place that many pads on the tray.
2. Take about 10 photos per count, changing things each time: flat vs folded pads, overlapping vs spread out, some pads dabbed with diluted red food coloring, straight above vs 45-degree angle, bright vs dim light, dark vs white background.
3. That gives about 160 photos. More variety matters more than more photos.
4. **Rename each photo with its count first**, then a letter: 00_a.jpg, 00_b.jpg ... 07_a.jpg ... 15_j.jpg. The number before the underscore **MUST** be the true count. Double-check counts; wrong labels teach the model wrong answers.
5. Resize if photos are huge (over 3 MB each) to keep training fast. On a Mac: select all, Preview, Tools, Adjust Size, width 1280. On Windows: the free PowerToys Image Resizer.
6. Drag the photos into ~/or-sentinel/data/photos/ in the VS Code file explorer.

Also take 3 or 4 wide photos of your whole mock-OR setup for your slides and social posts.

4.3 Acted operating room audio (Person C, about 60 minutes)

Record 3 clips on a phone in a quiet room, 2 to 3 minutes each. Speak clearly, pause 2 seconds between lines. Export as .wav (or convert later, see below). Optionally play monitor beeps from Freesound quietly in the background while recording.

Clip 1 - or1.wav (normal surgery):

```
SURGEON: Let's do a time out.
NURSE: Patient is John Doe, date of birth confirmed.
SURGEON: Laparoscopic appendectomy. Right side, site marked and confirmed.
NURSE: No known allergies.
NURSE: Adding five sponges.
NURSE: Adding five sponges.
SURGEON: Scalpel please. (pause 5 seconds)
NURSE: Sponge out. (say this line 10 times, pausing 3 seconds each time)
NURSE: Counts are correct.
SURGEON: Okay, let's close.
```

Clip 2 - or2.wav (spoken count mismatch): same as clip 1, but say 'Sponge out' only 9 times. The system should BLOCK because 10 went in and 9 came out.

Clip 3 - or3.wav (skipped time-out): same as clip 1 but skip the first four lines entirely. The system should show CAUTION: time-out not done.

The **hidden-problem room** does not need its own clip. The simulator makes the tray photo show one fewer sponge than the nurse counted, so audio says everything matches but the camera catches it. That is your key demo moment.

```
# If your phone saved .m4a instead of .wav, convert on the Nano:
sudo apt-get install -y ffmpeg # audio conversion tool
ffmpeg -i or1.m4a -ar 16000 -ac 1 data/audio/or1.wav # 16 kHz mono = what Whisper expects
```

4.4 Test scenarios list (Person D, 20 minutes)

Make a simple spreadsheet (save it as bench/scenarios.csv) with one row per scenario: scenario name, which clip, whether it is the hidden-problem room, and the expected final status. You will use it to report 'mismatches caught'.

Example rows:

```
scenario,clip,hidden_problem,expected_status
normal,or1,no,SAFE
spoken_mismatch,or2,no,BLOCK
skipped_timeout,or3,no,CAUTION
hidden_tray_problem,or1,yes,BLOCK
```

5. Label the photos automatically

Who: Person A. **Time:** 5 minutes. The script reads the count from each filename, writes training labels, and keeps 20% of photos aside for honest testing.

File: app/config.py (create this first; every other script uses it)

```
"""config.py - every setting in one place. Change values here, not in other files."""
import os

# Folder paths (worked out automatically, so scripts run from any folder)
ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
DATA_DIR = os.path.join(ROOT, "data")
PHOTO_DIR = os.path.join(DATA_DIR, "photos")
AUDIO_DIR = os.path.join(DATA_DIR, "audio")
STATE_DIR = os.path.join(ROOT, "state")
OFFLINE_FLAG = os.path.join(STATE_DIR, "OFFLINE") # if this file exists, the cloud is "cut"

# Local models served by ZRT on the Nano
LLM_URL = os.getenv("LLM_URL", "http://localhost:8000/v1") # 27B reasoning model
VLM_URL = os.getenv("VLM_URL", "http://localhost:8001/v1") # vision model
LLM_MODEL = os.getenv("LLM_MODEL", "Inferact/Qwen3.8-27B-NVFP4") # exact name from /v1/models
VLM_MODEL = os.getenv("VLM_MODEL", "yourname/or-sentinel-vision")

# Cloud model (any OpenAI-compatible API). Leave CLOUD_URL empty to run fully offline.
CLOUD_URL = os.getenv("CLOUD_URL", "")
CLOUD_KEY = os.getenv("CLOUD_KEY", "")
CLOUD_MODEL = os.getenv("CLOUD_MODEL", "")

# Decision thresholds
CONFIDENCE_THRESHOLD = 0.7 # local answer below this confidence counts as "unsure"
LONG_SURGERY_MIN = 360 # surgeries longer than this add fatigue risk
REQUEST_TIMEOUT_S = 30 # give up on a model call after this many seconds

os.makedirs(STATE_DIR, exist_ok=True)
```

File: app/make_labels.py

```
"""make_labels.py - turns photo filenames into labels and splits 80% train / 20% test.

Photos must be named <count><anything>.jpg, for example 07_a.jpg = 7 sponges.
Run from anywhere: python app/make_labels.py
"""
import json
import random
from pathlib import Path

import config

QUESTION = "How many gauze sponges are on the tray? Answer with a number only."

photos = sorted(p for p in Path(config.PHOTO_DIR).iterdir()
                 if p.suffix.lower() in (".jpg", ".jpeg", ".png"))
if not photos:
    raise SystemExit(f"No photos found in {config.PHOTO_DIR}")

rows, skipped = [], []
for p in photos:
    try:
        count = int(p.stem.split("_")[0]) # "07_a" -> 7
    except ValueError:
        skipped.append(p.name)
        continue
    rows.append({"image": str(p), "question": QUESTION, "answer": str(count)})

random.seed(42) # same shuffle every run, so the test set never changes
random.shuffle(rows)
cut = int(len(rows) * 0.8)

for name, part in (("train", rows[:cut]), ("test", rows[cut:])):
    out = Path(config.DATA_DIR) / f"{name}.jsonl"
    with open(out, "w") as f:
        for r in part:
            f.write(json.dumps(r) + "\n")
```

```

print(f"{name}: {len(part)} examples -> {out}")

counts = sorted({int(r["answer"]) for r in rows})
print("counts covered:", counts)
if skipped:
    print("skipped (bad filename):", skipped)

cd ~/or-sentinel && source .venv/bin/activate
python app/make_labels.py          # creates data/train.jsonl and data/test.jsonl
head -3 data/train.jsonl          # look at 3 labels: image path, question, answer
wc -l data/train.jsonl data/test.jsonl # count examples (about 128 and 32)

```

Expected output: train: 128 examples, test: 32 examples, and a list of counts 0 to 15. If any file is listed as skipped, rename it so it starts with a number and an underscore.

TIP: Never move photos between train and test after this. The test set must stay unseen by training so your accuracy chart is honest.

6. Fine-tune the vision model with HP's repo

Who: Person A. **Time:** 1 hour of setup, then training runs overnight. Start by Thursday 11pm. This is the riskiest step, so do it first.

WATCH OUT: This guide's author has not seen the inside of HP's script. Step 6.2 shows how to find where it loads data and swap in yours. Budget 30 minutes to read the script before changing anything.

6.1 Get HP's repo and test it

```
cd ~/Desktop
git clone https://github.com/jrgosalvez/Med_VLM_Fine-Tune_vLLM.git # download HP's example
cd Med_VLM_Fine-Tune_vLLM
chmod +x * # make scripts runnable (HP's PDF says to do this)
ls -la # see all files
cat README.md # read how it expects data and what stages exist
```

6.2 Find where it loads the medical data

```
grep -n -i "dataset\|load_dataset\|jsonl\|question\|answer\|image" *.sh *.py
# lists every line mentioning data; note file names + line numbers
grep -n "STAGE" finetune_qwen2_5_vl.sh # shows which stages exist (download, train, quantize, push)
```

You are looking for the place where it loads the medical question-answer dataset (often a Hugging Face dataset name, or a path to a JSON/JSONL file) and the field names it reads (for example image, question, answer). Your data/train.jsonl has exactly these three fields for every photo.

Three possible situations and what to do:

If the script...	Do this
reads a local JSON/JSONL file with image/question/answer fields	Change that path to the full path of your train.jsonl (run <code>realpath ~/or-sentinel/data/train.jsonl</code> to print it). Done.
reads a Hugging Face dataset by name	Upload your data as a Hugging Face dataset (below) and change the name to yours, OR replace that load line with a local JSONL load.
uses different field names (e.g. 'query' and 'label')	Rename the fields in your JSONL with the converter below.

```
# Converter: rename fields if HP's script expects different names.
# Edit the MAP line (left = our name, right = HP's name), then run: python rename_fields.py
import json, os
MAP = {"image": "image", "question": "question", "answer": "answer"} # e.g. {"question": "query"}
for name in ("train", "test"):
    src = os.path.expanduser(f"~/or-sentinel/data/{name}.jsonl")
    rows = [json.loads(l) for l in open(src)]
    with open(src.replace(".jsonl", "_hp.jsonl"), "w") as f:
        for r in rows:
            f.write(json.dumps({MAP[k]: v for k, v in r.items()}) + "\n")
    print("wrote", src.replace(".jsonl", "_hp.jsonl"))

# Option: upload your photos + labels as a Hugging Face dataset
pip install datasets huggingface_hub
python - << 'EOF'
import os
from datasets import load_dataset, Image
d = os.path.expanduser("~/or-sentinel/data")
ds = load_dataset("json", data_files={"train": f"{d}/train.jsonl", "test": f"{d}/test.jsonl"})
ds = ds.cast_column("image", Image()) # turns file paths into real images
ds.push_to_hub("<your-hf-username>/or-sponge-counts") # uses your HF_TOKEN
print(ds)
EOF
```

Before editing, back up the original script:

```
cp finetune_qwen2_5_vl.sh finetune_qwen2_5_vl.sh.original # backup; restore with cp the other way
```

6.3 Start training in tmux

```
tmux new -s train # a terminal that survives laptop disconnects
cd ~/Desktop/Med_VLM_Fine-Tune_vLLM
STAGE=all QFORMAT=nvfp4 bash finetune_qwen2_5_vl.sh 2>&1 | tee train.log
# STAGE=all download the base model, train, compress and upload in one go
# QFORMAT=nvfp4 compress the finished model to NVFP4 so it runs fast on the Nano
# tee train.log also saves every line of output to train.log
```

Leave it running: press **Ctrl+B**, release, then press **D**. Reconnect later with `tmux attach -t train`.

6.4 Check training is healthy

```
tail -f ~/Desktop/Med_VLM_Fine-Tune_vLLM/train.log # watch progress (Ctrl+C to stop watching)
nvidia-smi # GPU should show high utilization
free -h # memory in use
```

You see	Meaning	Action
loss numbers slowly going down	Training is working	Go to sleep
'Can't load model' or download errors	Base model not downloaded	Run <code>zrt pull Qwen/Qwen2.5-VL-7B-Instruct</code> , then restart the script
KeyError with a field name	Your field names don't match	Use the converter in 6.2
CUDA out of memory	Something else uses the GPU	<code>./stop.sh</code> to stop served models while training
FlashInfer error	Library version mismatch (HP's PDF mentions this)	Paste the full error into an AI assistant; often fixed by an environment variable or version pin
loss stays flat or jumps wildly	Bad labels or learning rate	Check a few labels; reduce epochs/learning rate if the script exposes them

6.5 When it finishes

The script uploads the model to your `HF_REPO_ID` (if it isn't set, it saves locally). Open your Hugging Face profile and confirm the model is there. Its name is what you put in `start.sh` and `config.py`, for example `<your-hf-username>/or-sentinel-vision`.

WATCH OUT: DEADLINE RULE: if training has not finished by Friday 10am, stop it and continue with the base model Qwen/Qwen2.5-VL-7B-Instruct. Report honestly: 'fine-tuning did not converge in time; here is the base model result.' A working demo beats a half-finished model.

7. Serve both models with ZRT

Who: Person B. **Time:** 30 minutes (mostly waiting for downloads). Each model becomes a web service: the 27B model on port 8000, the vision model on port 8001.

7.1 Download the models

```
zrt pull Inferact/Qwen3.8-27B-NVFP4          # the 27B reasoning model (name from HP's PDF)
zrt pull Qwen/Qwen2.5-VL-7B-Instruct         # base vision model (use until fine-tuning is done)
zrt pull <your-hf-username>/or-sentinel-vision # your fine-tuned model, after Part 6
zrt models                                  # confirm they are listed
```

7.2 start.sh and stop.sh

Save these in `~/or-sentinel/`. Edit the two model names at the top of `start.sh`. Before fine-tuning finishes, set VLM to the base model.

File: start.sh

```
#!/bin/bash
# start.sh - starts both models with ZRT, each in its own tmux session.
# Edit the two model names below to match yours.
LLM="Inferact/Qwen3.8-27B-NVFP4"
VLM="yourname/or-sentinel-vision"      # before fine-tuning: Qwen/Qwen2.5-VL-7B-Instruct

if ! tmux has-session -t llm 2>/dev/null; then
    tmux new -d -s llm "zrt serve $LLM --host 0.0.0.0 --port 8000 \
        --gpu-memory-utilization 0.50 --max-model-len 8192 2>&1 | tee llm.log"
    echo "Started 27B model (tmux session: llm, log: llm.log)"
else
    echo "27B model already running"
fi

if ! tmux has-session -t vlm 2>/dev/null; then
    tmux new -d -s vlm "zrt serve $VLM --host 0.0.0.0 --port 8001 \
        --gpu-memory-utilization 0.25 --max-model-len 4096 2>&1 | tee vlm.log"
    echo "Started vision model (tmux session: vlm, log: vlm.log)"
else
    echo "Vision model already running"
fi

echo "Loading takes a few minutes. Check with: curl localhost:8000/v1/models"
```

File: stop.sh

```
#!/bin/bash
# stop.sh - stops both models.
tmux kill-session -t llm 2>/dev/null && echo "Stopped 27B model"
tmux kill-session -t vlm 2>/dev/null && echo "Stopped vision model"
```

Flag	Plain meaning	Change it when
<code>--host 0.0.0.0</code>	Accept requests from other computers, not only the Nano	Never
<code>--port 8000</code>	The door number your code knocks on	Something else already uses that port
<code>--gpu-memory-utilization 0.50</code>	Use at most half of the memory, leaving room for the other model	Out-of-memory errors: lower both numbers (e.g. 0.40 and 0.20)
<code>--max-model-len 8192</code>	Longest prompt plus answer allowed; smaller saves memory	Out-of-memory: lower to 4096

```
cd ~/or-sentinel
chmod +x start.sh stop.sh          # make them runnable
./start.sh                        # start both models (takes a few minutes to load)
tmux attach -t llm                 # watch the 27B load; Ctrl+B then D to leave
tail -f llm.log                   # or watch the log file instead
```

WATCH OUT: HP's PDF shows the model written both as hf:<model> and plain <model>. If zrt serve says it can't find the model, try the other form. If zrt serve rejects a flag, run zrt serve --help to see the accepted form; HP's PDF says vLLM flags pass straight through.

WATCH OUT: During overnight training, run only the 27B model (comment out the vision block in start.sh by putting # in front of its lines) so training isn't slowed down.

7.3 Check the services are up

```
curl http://localhost:8000/v1/models # prints JSON with the 27B model's exact name
curl http://localhost:8001/v1/models # prints JSON with the vision model's exact name
```

Copy the exact id values from those two outputs into app/config.py (LLM_MODEL and VLM_MODEL). This is the most common cause of '404 model not found' errors.

```
curl http://localhost:8000/v1/chat/completions \
-H 'Content-Type: application/json' \
-d '{"model": "<id from /v1/models>", "messages": [{"role": "user", "content": "Reply with the word ready."}]}'
# expected: JSON containing "ready" somewhere in "content"
```

Test the vision model with a real photo using these two files:

File: app/vision.py

```
"""vision.py - asks the fine-tuned vision model to count sponges in one photo."""
import base64
import re
import time

from openai import OpenAI

import config

vlm = OpenAI(base_url=config.VLM_URL, api_key="none", timeout=config.REQUEST_TIMEOUT_S)
QUESTION = "How many gauze sponges are on the tray? Answer with a number only."

def count_tray(path):
    """Return (count or None, seconds taken). Raises an error if the model is down."""
    mime = "image/png" if str(path).lower().endswith(".png") else "image/jpeg"
    with open(path, "rb") as f:
        img = base64.b64encode(f.read()).decode()
    start = time.time()
    reply = vlm.chat.completions.create(
        model=config.VLM_MODEL, temperature=0, max_tokens=5,
        messages=[{"role": "user", "content": [
            {"type": "image_url", "image_url": {"url": f"data:{mime};base64,{img}"},
            {"type": "text", "text": QUESTION}]]])
    match = re.search(r"\d+", reply.choices[0].message.content or "")
    return (int(match.group(0)) if match else None, time.time() - start)
```

File: app/test_vision.py

```
"""test_vision.py - sends one tray photo to the vision model and prints its count.

Usage: python app/test_vision.py data/photos/07_a.jpg
"""
import sys

import vision

if len(sys.argv) != 2:
    raise SystemExit("Usage: python app/test_vision.py <photo path>")

count, seconds = vision.count_tray(sys.argv[1])
print(f"Model says: {count} sponges (took {seconds:.2f} s)")

cd ~/or-sentinel && source .venv/bin/activate
python app/test_vision.py data/photos/07_a.jpg # expected: "Model says: 7 sponges" (or close)
```

TIP: Test from your laptop too (not only the Nano) to prove the network path works: on your laptop run `curl http://<Nano Tailscale IP>:8000/v1/models`.

8. Turn the audio into timestamped text

Who: Person C. **Time:** 15 minutes. Each clip is transcribed once; the simulator then replays the text with the original timing, as if it were live. This is more reliable for a demo than live microphones.

File: `app/transcribe.py`

```
"""transcribe.py - turns an OR audio clip into timestamped lines of text.

Usage: python app/transcribe.py data/audio/or1.wav -> creates data/audio/or1.json
Runs on the CPU on purpose: GPU support for this library on ARM can be unreliable.
"""
import json
import sys
from pathlib import Path

from faster_whisper import WhisperModel

if len(sys.argv) != 2:
    raise SystemExit("Usage: python app/transcribe.py <audio.wav>")

audio = Path(sys.argv[1])
# small.en = good English accuracy on CPU. int8 = smaller and faster.
model = WhisperModel("small.en", device="cpu", compute_type="int8")
segments, _ = model.transcribe(str(audio), vad_filter=True) # vad_filter skips silence

lines = [{"t": round(s.start, 1), "text": s.text.strip()} for s in segments if s.text.strip()]
out = audio.with_suffix(".json")
with open(out, "w") as f:
    json.dump(lines, f, indent=1)
print(f"{len(lines)} lines -> {out}")
for line in lines[:5]:
    print(f" {line['t']:>6}s {line['text']}")

cd ~/or-sentinel && source .venv/bin/activate
python app/transcribe.py data/audio/or1.wav # creates data/audio/or1.json
python app/transcribe.py data/audio/or2.wav
python app/transcribe.py data/audio/or3.wav
cat data/audio/or1.json # check the words
```

Check the text carefully. If Whisper wrote 'sponge' as 'spunge' or 'five' as '5', that's fine for '5' (the Counter agent understands both) but fix obvious mistakes by editing the JSON in VS Code. The first run downloads the Whisper model (a few hundred MB).

WATCH OUT: The Nano's CPU is ARM-based. The script deliberately runs on the CPU because GPU support for faster-whisper on ARM can be unreliable. If even CPU mode fails to install, run `transcribe.py` on a laptop (same commands, after `pip install faster-whisper`) and copy the `.json` files into `data/audio/` on the Nano. Say so in the README.

9. Build the app

Who: Person B (agents, fusion, router, simulator) and Person C (dashboard in Part 10). Create each file below in ~/or-sentinel/app/. Test with one room before eight.

9.1 agents.py - the four AI agents

Counter and Checklist run at the same time on every line of speech. Each prompt includes examples, which makes the model's JSON far more reliable. The `_extract_json` helper strips hidden reasoning (text between `<think>` tags) and any extra words around the JSON.

File: app/agents.py

```
"""agents.py - the four AI agents. All run on the local 27B model through ZRT.

Counter      : reads one line of speech, returns sponges added / removed
Checklist    : reads one line of speech, returns a WHO checklist event
Critic       : explains the rules engine's decision to the nurse in one sentence
Scribe       : writes the final surgical safety log
"""

import json
import re
from concurrent.futures import ThreadPoolExecutor

from openai import OpenAI

import config

local = OpenAI(base_url=config.LLM_URL, api_key="none", timeout=config.REQUEST_TIMEOUT_S)
pool = ThreadPoolExecutor(max_workers=16)

COUNTER = """You track surgical sponge counts from operating room speech.
Read ONE line. Reply ONLY with JSON: {"added": int, "removed": int}.
added = sponges opened onto the sterile field.
removed = sponges taken off the field and counted.
Examples:
"Adding five sponges." -> {"added": 5, "removed": 0}
"Sponge out." -> {"added": 0, "removed": 1}
"Two sponges out." -> {"added": 0, "removed": 2}
"Suction please." -> {"added": 0, "removed": 0}"""

CHECKLIST = """You track the WHO Surgical Safety Checklist from operating room speech.
Read ONE line. Reply ONLY with JSON: {"event": E, "detail": "short text"}.
E must be one of: time_out, patient_confirmed, site_confirmed, allergies_confirmed,
count_confirmed, close_requested, none.
Examples:
"Let's do a time out." -> {"event": "time_out", "detail": ""}
"Patient is John Doe, date of birth confirmed." -> {"event": "patient_confirmed", "detail": ""}
"Left knee, site marked and confirmed." -> {"event": "site_confirmed", "detail": "left knee"}
"No known allergies." -> {"event": "allergies_confirmed", "detail": "none"}
"Counts are correct." -> {"event": "count_confirmed", "detail": ""}
"Okay, let's close." -> {"event": "close_requested", "detail": ""}
"Scalpel please." -> {"event": "none", "detail": ""}"""

CRITIC = """You are a surgical safety critic. The rules engine produced the status and alerts below.
In under 25 words, tell the circulating nurse what is wrong and exactly what to check.
If the status is SAFE, say so briefly.
Reply ONLY with JSON: {"message": "...}"""

SCRIBE = """You write a short surgical safety log from a list of events.
Use plain sentences with times. Include checklist steps completed, final counts and any alerts."""

def _extract_json(text):
    """Pull the first {...} block out of a model reply."""
    text = re.sub(r"<think>.*?</think>", "", text or "", flags=re.S) # drop hidden reasoning
    match = re.search(r"\{.*\}", text, re.S)
    if not match:
        return {}
    try:
        return json.loads(match.group(0))
    except json.JSONDecodeError:
```

```

        return {}

def ask(system, user, max_tokens=200):
    """Ask the local 27B model and return its JSON reply as a dict ({} on failure)."""
    reply = local.chat.completions.create(
        model=config.LLM_MODEL, temperature=0, max_tokens=max_tokens,
        messages=[{"role": "system", "content": system},
                  {"role": "user", "content": user}])
    return _extract_json(reply.choices[0].message.content)

def _safe_int(value):
    try:
        return max(int(value), 0)
    except (TypeError, ValueError):
        return 0

def process_line(text):
    """Run the Counter and Checklist agents at the same time on one line of speech."""
    counter_job = pool.submit(ask, COUNTER, text)
    checklist_job = pool.submit(ask, CHECKLIST, text)
    c = counter_job.result()
    k = checklist_job.result()
    counts = {"added": _safe_int(c.get("added")), "removed": _safe_int(c.get("removed"))}
    event = k.get("event", "none")
    if event not in ("time_out", "patient_confirmed", "site_confirmed",
                    "allergies_confirmed", "count_confirmed", "close_requested"):
        event = "none"
    return counts, {"event": event, "detail": k.get("detail", "")}

def critic(state, status, alerts):
    info = json.dumps({"status": status, "alerts": alerts,
                      "sponges_in": state["added"], "sponges_out": state["removed"],
                      "camera_count": state["vision_count"]})
    return ask(CRITIC, info).get("message", "")

def scribe(events):
    reply = local.chat.completions.create(
        model=config.LLM_MODEL, temperature=0, max_tokens=600,
        messages=[{"role": "system", "content": SCRIBE},
                  {"role": "user", "content": json.dumps(events)}])
    text = reply.choices[0].message.content or ""
    return re.sub(r"<think>.*?</think>", "", text, flags=re.S).strip()

cd ~/or-sentinel/app && source ../.venv/bin/activate
python -c "import agents; print(agents.process_line('Adding five sponges.'))"
# expected: ({'added': 5, 'removed': 0}, {'event': 'none', 'detail': ''})
python -c "import agents; print(agents.process_line('Okay, let's close.'))"
# expected: ({'added': 0, 'removed': 0}, {'event': 'close_requested', 'detail': ''})

```

TIP: If replies are slow because the 27B model 'thinks' out loud first, add '/no_think' to the end of each user message in ask() (works for many Qwen3-family models), or ask HP mentors which reasoning flag to pass to zrt serve.

9.2 fusion.py - the safety rules

Plain rules decide the status. BLOCK only happens when the surgeon asks to close AND either the spoken counts don't balance or the camera disagrees with the nurse's count.

File: app/fusion.py

```

"""fusion.py - compares what was HEARD with what was SEEN.

These fixed rules, not the AI, decide whether closing is blocked.
The AI only reads speech, counts photos and explains.
"""
import time

import config

```

```

def evaluate(s):
    """Return (status, alerts, risk 0-100) for one operating room state dict."""
    alerts, risk = [], 0
    missing = s["added"] - s["removed"]
    cam = s.get("vision_count")
    camera_disagrees = cam is not None and cam != s["removed"]

    if camera_disagrees:
        alerts.append(f"Nurse counted {s['removed']} out, camera sees {cam} on the tray")
        risk += 40
    if "time_out" not in s["checklist"]:
        alerts.append("Time-out not done yet")
        risk += 15
    if s["close_requested"] and missing != 0:
        alerts.append(f"Count mismatch: {s['added']} in, {s['removed']} out")
        risk += 45
    minutes = (time.time() - s["started"]) / 60 * s.get("speed", 1)
    if minutes > config.LONG_SURGERY_MIN:
        alerts.append("Surgery over 6 hours: higher error risk")
        risk += 10

    if s["close_requested"] and (missing != 0 or camera_disagrees):
        status = "BLOCK"
    elif alerts:
        status = "CAUTION"
    else:
        status = "SAFE"
    return status, alerts, min(risk, 100)

```

Situation	Status	Risk added
Camera count differs from nurse's 'out' count	CAUTION (BLOCK if closing)	+40
Time-out not done yet	CAUTION	+15
Closing requested while in != out	BLOCK	+45
Surgery longer than 6 hours (simulated time)	CAUTION	+10

9.3 ledger.py - tamper-proof log

Each entry stores the fingerprint (SHA-256 hash) of the previous entry. Changing any old entry changes its fingerprint and breaks the chain, and `verify()` returns the entry number where it broke. A file lock stops two programs writing at the same instant.

File: app/ledger.py

```

"""ledger.py - tamper-proof log. Each entry includes the hash of the one before it,
so editing any old entry breaks the chain and verify() reports where."""
import fcntl
import hashlib
import json
import os
import time

import config

LOG = os.path.join(config.STATE_DIR, "ledger.jsonl")

def _hash(entry):
    return hashlib.sha256(json.dumps(entry, sort_keys=True).encode()).hexdigest()

def record(entry):
    """Append one entry. A file lock stops two processes writing at the same moment."""
    with open(LOG, "a+") as f:
        fcntl.flock(f, fcntl.LOCK_EX)
        f.seek(0)
        lines = f.read().splitlines()
        prev = json.loads(lines[-1])["hash"] if lines else "GENESIS"
        entry = {"time": round(time.time(), 3), **entry, "prev": prev}

```

```

        entry["hash"] = _hash(entry)
        f.write(json.dumps(entry) + "\n")
        fcntl.flock(f, fcntl.LOCK_UN)

def read_all():
    if not os.path.exists(LOG):
        return []
    out = []
    with open(LOG) as f:
        for line in f:
            try:
                out.append(json.loads(line))
            except json.JSONDecodeError:
                pass
    return out

def verify():
    """Return None if the log is untouched, otherwise the number of the first bad entry."""
    if not os.path.exists(LOG):
        return None
    prev = "GENESIS"
    with open(LOG) as f:
        for i, line in enumerate(f, 1):
            try:
                e = json.loads(line)
                h = e.pop("hash")
            except (json.JSONDecodeError, KeyError):
                return i
            if e.get("prev") != prev or _hash(e) != h:
                return i
            prev = h
    return None

```

9.4 cloud_router.py - the local vs cloud decision

This is the challenge's core requirement: an explicit, defensible, measurable escalation rule. The local model always answers first and rates its own confidence. The question goes to the cloud only if ALL of these are true: not urgent, local confidence below 0.7, cloud reachable, and Presidio finds no personal details left after stripping. Every decision is written to the ledger, which is where your escalation-rate metric comes from.

File: app/cloud_router.py

```

"""cloud_router.py - decides LOCAL vs CLOUD for protocol questions.

Rule: anything urgent, anything the local model is confident about, or anything while
the cloud is offline stays LOCAL. Otherwise patient data is stripped, double-checked,
and only then is the anonymous question sent to the cloud.
"""
import os

from openai import OpenAI

import agents
import config
import ledger

LOCAL_SYS = """Answer the surgical protocol question briefly and practically.
Reply ONLY with JSON: {"answer": "...", "confidence": number from 0 to 1}
confidence = how sure you are that the answer is correct and complete."""

_engines = {}

def _presidio():
    """Load Presidio once, the first time it is needed (it takes a few seconds)."""
    if not _engines:
        from presidio_analyzer import AnalyzerEngine
        from presidio_anonymizer import AnonymizerEngine
        _engines["a"], _engines["z"] = AnalyzerEngine(), AnonymizerEngine()
    return _engines["a"], _engines["z"]

```

```

def strip_phi(text):
    """Replace names, dates, phone numbers, IDs and so on with tags like <PERSON>."""
    analyzer, anonymizer = _presidio()
    found = analyzer.analyze(text=text, language="en")
    return anonymizer.anonymize(text=text, analyzer_results=found).text

def still_has_phi(text):
    analyzer, _ = _presidio()
    return bool(analyzer.analyze(text=text, language="en"))

def cloud_is_offline():
    return os.path.exists(config.OFFLINE_FLAG) or not config.CLOUD_URL

def answer(question, urgent=False):
    """Return {"route", "answer", "sent", "confidence"}. "sent" is exactly what left the Nano."""
    local = agents.ask(LOCAL_SYS, question, max_tokens=300)
    try:
        conf = float(local.get("confidence", 0))
    except (TypeError, ValueError):
        conf = 0.0
    local_answer = local.get("answer", "")

    def stay_local(reason):
        ledger.record({"type": "route", "route": "local", "reason": reason,
                      "confidence": conf, "bytes_sent": 0})
        return {"route": f"local ({reason})", "answer": local_answer, "sent": "", "confidence": conf}

    if urgent:
        return stay_local("urgent")
    if conf >= config.CONFIDENCE_THRESHOLD:
        return stay_local("confident")
    if cloud_is_offline():
        return stay_local("cloud offline")

    clean = strip_phi(question)
    if still_has_phi(clean):
        # second check: never send if anything is left
        return stay_local("patient data remained")

    try:
        cloud = OpenAI(base_url=config.CLOUD_URL, api_key=config.CLOUD_KEY, timeout=15)
        reply = cloud.chat.completions.create(
            model=config.CLOUD_MODEL, max_tokens=300,
            messages=[{"role": "user", "content": clean}])
        ledger.record({"type": "route", "route": "cloud", "reason": "low confidence",
                      "confidence": conf, "bytes_sent": len(clean.encode())})
        return {"route": "cloud", "answer": reply.choices[0].message.content,
                "sent": clean, "confidence": conf}
    except Exception:
        # network cut mid-call -> fall back
        return stay_local("cloud unreachable")

```

Setting up a cloud model (optional but recommended): any OpenAI-compatible provider works. Put its details in `~/.bashrc` (never in code):

```

echo 'export CLOUD_URL="https://<provider-base-url>/v1"' >> ~/.bashrc
echo 'export CLOUD_KEY="<your-api-key>"' >> ~/.bashrc
echo 'export CLOUD_MODEL="<provider-model-name>"' >> ~/.bashrc
source ~/.bashrc

```

If you have no cloud key, leave `CLOUD_URL` empty. The system then shows 'cloud offline' and stays local, which still demonstrates the decision logic. Your privacy split-screen will show what **WOULD** be sent via the 'sent' field only when a cloud is configured.

```

python -c "import cloud_router as c; print(c.strip_phi('Patient Maria Lopez, DOB 03/14/1971, MRN 4471923'))"
# expected: names and dates replaced, e.g. 'Patient <PERSON>, DOB <DATE_TIME>, MRN ...'

```

WATCH OUT: Presidio catches most names and dates but can miss unusual names or record numbers. Test with the exact demo sentence. The second check (`still_has_phi`) blocks the cloud call if anything is still detected, so a miss means 'stay local', not 'leak'.

9.5 simulate.py - 8 rooms at once

Each room is a separate thread reading a transcript with its original timing (sped up). When a sponge comes out, the room picks a real tray photo showing the right number of sponges and asks the vision model to count it. In the hidden-problem room, the photo shows one fewer sponge than the nurse counted. Status is saved to state/or<N>.json every line.

File: app/simulate.py

```
"""simulate.py - replays several operating rooms at the same time.
```

```
Each room reads a transcript (data/audio/*.json), runs the agents on every line,
checks the tray photo with the vision model when a sponge comes out, and saves its
live status to state/or<N>.json for the dashboard.
```

```
Usage: python app/simulate.py --ors 8 --speed 4 --problem 6
```

```
--ors      how many rooms to run at once
```

```
--speed    4 = four times faster than the real recording
```

```
--problem  which room secretly has one sponge missing from the tray (0 = none)
```

```
"""
```

```
import argparse
import glob
import json
import os
import random
import threading
import time
from pathlib import Path
```

```
import agents
import config
import fusion
import ledger
import vision
```

```
PHOTOS = [p for p in glob.glob(os.path.join(config.PHOTO_DIR, ""))
            if p.lower().endswith((".jpg", ".jpeg", ".png"))]
```

```
def photo_with(count):
    """Pick a random tray photo that shows exactly this many sponges (None if none exist)."""
    hits = []
    for p in PHOTOS:
        try:
            if int(Path(p).stem.split("_")[0]) == count:
                hits.append(p)
        except ValueError:
            pass
    return random.choice(hits) if hits else None
```

```
def save(s):
    path = os.path.join(config.STATE_DIR, f"or{s['or']}.json")
    tmp = path + ".tmp"
    with open(tmp, "w") as f:
        json.dump(s, f)
    os.replace(tmp, path)      # swap in one step, so the dashboard never reads half a file
```

```
def run_or(or_id, script, speed, hidden_problem):
    s = {"or": or_id, "script": os.path.basename(script), "started": time.time(), "speed": speed,
         "added": 0, "removed": 0, "vision_count": None, "mode": "audio + camera",
         "checklist": [], "close_requested": False, "status": "SAFE", "alerts": [],
         "risk": 0, "message": "", "transcript": [], "latencies": [], "events": [],
         "done": False}
    with open(script) as f:
        lines = json.load(f)
    start = time.time()
    for line in lines:
        wait = line["t"] / speed - (time.time() - start)  # keep the recording's timing
        if wait > 0:
            time.sleep(wait)
        t0 = time.time()
```

```

try:
    counts, check = agents.process_line(line["text"])
except Exception as e:
    # 27B model down or slow
    s["message"] = f"Language model unavailable: {type(e).__name__}"
    save(s)
    continue
s["transcript"] = (s["transcript"] + [line["text"]])[-6:]
s["added"] += counts["added"]
s["removed"] += counts["removed"]
if check["event"] != "none":
    s["checklist"].append(check["event"])
    s["events"].append({"t": line["t"], "event": check["event"], "detail": check["detail"]})
    if check["event"] == "close_requested":
        s["close_requested"] = True
if counts["added"] or counts["removed"]:
    s["events"].append({"t": line["t"], "sponges_in": s["added"],
                        "sponges_out": s["removed"]})

if counts["removed"]:
    # a sponge came out -> camera checks tray
    on_tray = s["removed"] - (1 if hidden_problem else 0)
    photo = photo_with(max(on_tray, 0))
    if photo:
        try:
            s["vision_count"], _ = vision.count_tray(photo)
            s["mode"] = "audio + camera"
        except Exception:
            s["vision_count"] = None
            s["mode"] = "AUDIO ONLY - camera agent down"

status, alerts, risk = fusion.evaluate(s)
if status != s["status"]:
    try:
        s["message"] = agents.critic(s, status, alerts)
    except Exception:
        s["message"] = "; ".join(alerts)
    ledger.record({"type": "status", "or": or_id, "status": status, "alerts": alerts,
                  "sponges_in": s["added"], "sponges_out": s["removed"],
                  "camera": s["vision_count"]})
    s.update(status=status, alerts=alerts, risk=risk)
    s["latencies"].append(round(time.time() - t0, 3))
    save(s)
s["done"] = True
save(s)

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--ors", type=int, default=8)
    ap.add_argument("--speed", type=float, default=4)
    ap.add_argument("--problem", type=int, default=6)
    a = ap.parse_args()

    scripts = sorted(glob.glob(os.path.join(config.AUDIO_DIR, "*.json")))
    if not scripts:
        raise SystemExit("No transcripts in data/audio/. Run app/transcribe.py first.")
    if not PHOTOS:
        print("WARNING: no tray photos found, camera checks will be skipped")
    for old in glob.glob(os.path.join(config.STATE_DIR, "or*.json")):
        os.remove(old)
        # clear the previous run's rooms

    threads = [threading.Thread(target=run_or,
                                args=(i, scripts[(i - 1) % len(scripts)], a.speed,
                                       i == a.problem))
               for i in range(1, a.ors + 1)]
    print(f"Running {a.ors} rooms at {a.speed}x speed. "
          f"Hidden problem in room {a.problem or 'none'}.")
    for t in threads:
        t.start()
    for t in threads:
        t.join()
    print("All rooms finished.")

if __name__ == "__main__":
    main()

```

```
cd ~/or-sentinel && source .venv/bin/activate
python app/simulate.py --ors 1 --speed 4 --problem 0      # one normal room first
cat state/or1.json                                       # expect "status": "SAFE" at the end
python app/simulate.py --ors 3 --speed 4 --problem 0      # rooms 1-3 use clips or1, or2, or3
# expect: room 1 SAFE, room 2 BLOCK (spoken mismatch), room 3 CAUTION (no time-out)
python app/simulate.py --ors 8 --speed 4 --problem 6      # full demo; room 6 hides a missing sponge
```

Rooms reuse clips in order (room 4 uses or1 again, and so on). With --problem 6, room 6 plays clip 3's pattern plus the hidden tray problem; choose the problem room so it uses clip or1 (rooms 1, 4 or 7 with three clips) if you want the cleanest 'audio says fine, camera says no' moment.

TIP: Speed 4 means a 3-minute clip finishes in 45 seconds. For the live demo use speed 2 to 4. For benchmarks use 8 to push the Nano harder.

10. Dashboard

Who: Person C. **Time:** 1 to 2 hours including styling. The top section refreshes itself every second (Streamlit's fragment feature, version 1.37 or newer). Below it: a cloud on/off switch, the privacy split-screen, and the Scribe agent's surgical log.

File: `app/dashboard.py`

```
"""dashboard.py - the surgical floor command center (Streamlit).

Run: streamlit run app/dashboard.py --server.address 0.0.0.0 --server.port 8501
"""
import glob
import json
import os

import streamlit as st

import agents
import cloud_router
import config
import ledger

st.set_page_config(page_title="OR Sentinel", layout="wide")
st.title("OR Sentinel - Surgical Floor Command Center")
st.caption("All AI runs on one HP ZGX Nano. Patient data never leaves the hospital.")

COLORS = {"SAFE": "#1e8e3e", "CAUTION": "#e37400", "BLOCK": "#d93025"}

def load_rooms():
    rooms = []
    paths = glob.glob(os.path.join(config.STATE_DIR, "or*.json"))
    for path in sorted(paths, key=lambda p: int(os.path.basename(p)[2:-5])):
        try:
            with open(path) as f:
                rooms.append(json.load(f))
        except (json.JSONDecodeError, OSError):
            pass
    return rooms

@st.fragment(run_every=1)          # this part refreshes itself every second
def live_floor():
    rooms = load_rooms()
    if not rooms:
        st.info("No rooms running. Start: python app/simulate.py --ors 8 --speed 4 --problem 6")
        return

    cols = st.columns(4)
    for i, r in enumerate(rooms):
        cam = r["vision_count"] if r["vision_count"] is not None else "-"
        with cols[i % 4]:
            st.markdown(
                f"<div style='background:{COLORS[r['status']]};color:white;padding:12px;"
                f"border-radius:10px;margin-bottom:6px'">
                f"<div style='font-size:20px'"><b>OR {r['or']} &nbsp; {r['status']}</b></div>"
                f"Sponges in {r['added']} &middot; out {r['removed']} &middot; camera {cam}<br>"
                f"Risk {r['risk']}% &middot; {r['mode']}</div>", unsafe_allow_html=True)
            for a in r["alerts"]:
                st.caption(a)
            if r["message"]:
                st.write(f"Critic: {r['message']}")
            with st.expander("Live transcript"):
                for line in r["transcript"]:
                    st.write(line)

    lat = sorted(x for r in rooms for x in r["latencies"])
    routes = [e for e in ledger.read_all() if e.get("type") == "route"]
    cloud_calls = sum(1 for e in routes if e["route"] == "cloud")
    sent = sum(e.get("bytes_sent", 0) for e in routes)
    c1, c2, c3, c4 = st.columns(4)
    c1.metric("Rooms live", sum(1 for r in rooms if not r["done"]))
```

```

if lat:
    c2.metric("p95 latency per line", f"{lat[max(int(len(lat) * 0.95) - 1, 0)]:.2f} s")
c3.metric("Cloud escalations", f"{cloud_calls} of {len(routes)}")
c4.metric("Anonymous bytes sent to cloud", sent)

bad = ledger.verify()
if bad:
    st.error(f"RECORD TAMPERED at entry #{bad}")
else:
    st.success("Surgical record intact (hash chain verified)")

live_floor()

st.divider()
left, right = st.columns([1, 3])
with left:
    st.subheader("Cloud link")
    if cloud_router.cloud_is_offline():
        st.error("Cloud OFFLINE - everything runs locally")
        if st.button("Reconnect cloud"):
            if os.path.exists(config.OFFLINE_FLAG):
                os.remove(config.OFFLINE_FLAG)
            st.rerun()
    else:
        st.success("Cloud available (anonymous questions only)")
        if st.button("Cut cloud link"):
            open(config.OFFLINE_FLAG, "w").close()
            st.rerun()

with right:
    st.subheader("Ask a protocol question")
    q = st.text_input("Question (try including a fake patient name and date of birth)",
        "Patient Maria Lopez, DOB 03/14/1971, MRN 4471923, left laparoscopic "
        "appendectomy. Which counts are required before closing?")
    urgent = st.checkbox("Urgent (time-critical)")
    if st.button("Ask"):
        with st.spinner("Deciding local vs cloud..."):
            res = cloud_router.answer(q, urgent=urgent)
        a, b = st.columns(2)
        a.markdown("***Stayed on the Nano**")
        a.code(q, wrap_lines=True)
        b.markdown(f"***Sent to cloud** - route: `{res['route']}`")
        b.code(res["sent"] or "Nothing left the building", wrap_lines=True)
        st.write(f"Local confidence: {res['confidence']:.2f}")
        st.write(res["answer"])

st.divider()
st.subheader("Surgical log (Scribe agent)")
rooms = load_rooms()
if rooms:
    pick = st.selectbox("Room", [r["or"] for r in rooms])
    if st.button("Write log"):
        room = next(r for r in rooms if r["or"] == pick)
        with st.spinner("Scribe agent writing..."):
            st.markdown(agents.scribe(room["events"]))

# Terminal 1: rooms
cd ~/or-sentinel && source .venv/bin/activate
python app/simulate.py --ors 8 --speed 3 --problem 7

# Terminal 2: dashboard
cd ~/or-sentinel && source .venv/bin/activate
streamlit run app/dashboard.py --server.address 0.0.0.0 --server.port 8501

```

Open it on your laptop: VS Code usually pops up 'Your application running on port 8501 is available' with an **Open in Browser** button (this is port forwarding through your SSH connection). Otherwise open `http://<Nano Tailscale IP>:8501` in your laptop's browser.

What you should see: 8 tiles turning green, orange or red as the rooms play; alerts and the Critic's sentence under each tile; four numbers (rooms live, p95 latency, cloud escalations, bytes sent); a green 'Surgical record intact' bar.

11. Benchmarks and metrics

Who: Person D with Person A. **Time:** 2 to 3 hours. The brief requires benchmarks AND how and why they were chosen. Run everything from ~/or-sentinel with the venv active.

11.1 Vision accuracy: base vs fine-tuned

File: bench/eval_vision.py

```
"""eval_vision.py - measures sponge-count accuracy on the TEST photos only.

Usage:  VLM_MODEL="<model name>" python bench/eval_vision.py base
        VLM_MODEL="<model name>" python bench/eval_vision.py finetuned
"""
import json
import os
import sys

sys.path.insert(0, os.path.join(os.path.dirname(os.path.abspath(__file__)), "..", "app"))
import config # noqa: E402
import vision # noqa: E402

label = sys.argv[1] if len(sys.argv) > 1 else "model"
with open(os.path.join(config.DATA_DIR, "test.jsonl")) as f:
    rows = [json.loads(line) for line in f if line.strip()]
exact = close = 0
secs = []
for r in rows:
    guess, t = vision.count_tray(r["image"])
    truth = int(r["answer"])
    exact += guess == truth
    close += guess is not None and abs(guess - truth) <= 1
    secs.append(t)
    print(f"{os.path.basename(r['image']):<16} truth {truth:>2} guess {guess}")

res = {"model": label, "model_name": config.VLM_MODEL, "n": len(rows),
       "exact_pct": round(100 * exact / len(rows), 1),
       "within_1_pct": round(100 * close / len(rows), 1),
       "avg_seconds": round(sum(secs) / len(secs), 3)}
print(res)
out_dir = os.path.join(config.ROOT, "bench", "results")
os.makedirs(out_dir, exist_ok=True)
with open(os.path.join(out_dir, f"vision_{label}.json"), "w") as f:
    json.dump(res, f, indent=1)

# 1. Serve the BASE model on port 8001 (set VLM in start.sh to Qwen/Qwen2.5-VL-7B-Instruct)
tmux kill-session -t vlm; ./start.sh
curl localhost:8001/v1/models # copy its id
VLM_MODEL="<base id>" python bench/eval_vision.py base

# 2. Serve your FINE-TUNED model on port 8001 (set VLM back to your model)
tmux kill-session -t vlm; ./start.sh
curl localhost:8001/v1/models
VLM_MODEL="<fine-tuned id>" python bench/eval_vision.py finetuned
```

11.2 Latency at 1, 4 and 8 rooms

File: bench/scale.sh

```
#!/bin/bash
# scale.sh - runs the simulator with 1, 4 and 8 rooms and records latency for each.
cd "$(dirname "$0")/.."
rm -f bench/results/scale.csv
for N in 1 4 8; do
    echo "=== $N rooms ==="
    python app/simulate.py --ors $N --speed 8 --problem 0
    python bench/collect.py $N
done
echo "Done. Results in bench/results/scale.csv"
```

File: bench/collect.py

```
"""collect.py - reads every room's latencies and appends p50 / p95 for this room count."""
```

```

import csv
import glob
import json
import os
import sys

ROOT = os.path.join(os.path.dirname(os.path.abspath(__file__)), "..")
n = int(sys.argv[1])
lat = []
for path in glob.glob(os.path.join(ROOT, "state", "or*.json")):
    with open(path) as f:
        lat.extend(json.load(f)["latencies"])
lat.sort()
if not lat:
    raise SystemExit("No latencies found. Did the simulator run?")
p50 = lat[len(lat) // 2]
p95 = lat[max(int(len(lat) * 0.95) - 1, 0)]
out = os.path.join(ROOT, "bench", "results", "scale.csv")
os.makedirs(os.path.dirname(out), exist_ok=True)
new = not os.path.exists(out)
with open(out, "a", newline="") as f:
    w = csv.writer(f)
    if new:
        w.writerow(["rooms", "lines_processed", "p50_s", "p95_s"])
    w.writerow([n, len(lat), round(p50, 3), round(p95, 3)])
print(f"{n} rooms: {len(lat)} lines, p50 {p50:.3f} s, p95 {p95:.3f} s")

chmod +x bench/scale.sh
./bench/scale.sh # runs 1, then 4, then 8 rooms; writes bench/results/scale.csv
cat bench/results/scale.csv

```

While the 8-room run is going, open a second terminal and run `free -h` and `nvidia-smi`. Write down the memory used. On the GB10, CPU and GPU share memory, so `nvidia-smi` may say 'Not Supported' for memory; use the `free -h` number.

11.3 Cloud escalation rate

File: bench/questions.txt (20 questions; 5 contain fake patient details on purpose)

```

Which counts are required before closing a laparoscopic appendectomy?
What are the three phases of the WHO Surgical Safety Checklist?
Patient Maria Lopez, DOB 03/14/1971, left knee arthroscopy. What must be confirmed at time-out?
What should the team do if the sponge count is incorrect before closing?
Which items are usually counted during open abdominal surgery?
Patient John Chen, MRN 5520193, right hip replacement. Is a sharps count required?
What is a retained surgical item?
When should an intraoperative X-ray be considered for a missing sponge?
What does the circulating nurse confirm at sign-out?
Are counts required for a cesarean section, and when?
What is the purpose of the time-out before incision?
Patient Aisha Khan, age 42, thyroidectomy. Which allergies must be confirmed at sign-in?
How should counts be handled during a staff shift change mid-surgery?
What is the difference between radiopaque and non-radiopaque sponges?
Which surgical specialties most often skip instrument counts?
What documentation is required after a count discrepancy is resolved?
Patient Robert Diaz, DOB 07/02/1958, CABG. When are the counts performed?
What is wrong-site surgery and how is it prevented?
What should happen if the surgeon refuses to pause for a recount?
Summarize the sign-in checks before anesthesia induction.

```

File: bench/escalation_eval.py

```

"""escalation_eval.py - runs every question in bench/questions.txt through the router
and reports how many stayed local, how many went to the cloud, and bytes sent."""
import os
import sys

sys.path.insert(0, os.path.join(os.path.dirname(os.path.abspath(__file__)), "..", "app"))
import cloud_router # noqa: E402
import config # noqa: E402

with open(os.path.join(config.ROOT, "bench", "questions.txt")) as f:
    questions = [q.strip() for q in f if q.strip()]
    local = cloud = sent = 0

```

```

for q in questions:
    r = cloud_router.answer(q)
    if r["route"] == "cloud":
        cloud += 1
        sent += len(r["sent"].encode())
    else:
        local += 1
    print(f"[{r['route']:<28}] conf {r['confidence']:.2f} {q[:60]}")
print(f"\n{len(questions)} questions: {local} local, {cloud} cloud "
      f"({100 * cloud / len(questions):.0f}% escalation), {sent} anonymous bytes sent")

python bench/escalation_eval.py | tee bench/results/escalation.txt

```

11.4 Mismatches caught

Run each row of `bench/scenarios.csv` (Part 4.4) with the simulator and record the final status of each room. Report 'expected BLOCK scenarios caught: X of Y'. Example: `python app/simulate.py --ors 3 --speed 8 --problem 1` then `grep status state/or*.json`.

11.5 Charts for slides

File: `bench/charts.py`

```

"""charts.py - draws the two slide charts from the benchmark results."""
import csv
import json
import os

import matplotlib
matplotlib.use("Agg")          # draw to files, no screen needed
import matplotlib.pyplot as plt # noqa: E402

R = os.path.join(os.path.dirname(os.path.abspath(__file__)), "results")

with open(os.path.join(R, "scale.csv")) as f:
    rows = list(csv.DictReader(f))
    rooms = [int(r["rooms"]) for r in rows]
    plt.figure(figsize=(6, 4))
    plt.plot(rooms, [float(r["p95_s"]) for r in rows], marker="o", label="p95")
    plt.plot(rooms, [float(r["p50_s"]) for r in rows], marker="o", label="p50")
    plt.xlabel("Operating rooms running at once")
    plt.ylabel("Latency per speech line (seconds)")
    plt.title("One HP ZGX Nano, many operating rooms")
    plt.legend()
    plt.grid(alpha=0.3)
    plt.tight_layout()
    plt.savefig(os.path.join(R, "scale.png"), dpi=200)

with open(os.path.join(R, "vision_base.json")) as f:
    b = json.load(f)
with open(os.path.join(R, "vision_finetuned.json")) as f:
    ft = json.load(f)
plt.figure(figsize=(6, 4))
x = [0, 1]
plt.bar([i - 0.2 for i in x], [b["exact_pct"], ft["exact_pct"]], width=0.4, label="Exact count")
plt.bar([i + 0.2 for i in x], [b["within_1_pct"], ft["within_1_pct"]], width=0.4,
        label="Within 1")
plt.xticks(x, ["Base model", "Fine-tuned on Nano"])
plt.ylabel("Accuracy on test photos (%)")
plt.ylim(0, 100)
plt.title("Sponge counting: before vs after fine-tuning")
plt.legend()
plt.tight_layout()
plt.savefig(os.path.join(R, "vision.png"), dpi=200)
print("Saved bench/results/scale.png and bench/results/vision.png")

python bench/charts.py          # creates bench/results/scale.png and vision.png

```

11.6 Metrics table for README and slides (fill in your numbers)

Metric	How measured	Why we chose it
Sponge-count accuracy, base vs fine-tuned (exact and within 1)	eval_vision.py on 32 held-out test photos	Proves training on the Nano added value over an off-the-shelf model
Mismatches caught	Scenario list run through the simulator	The core safety claim: does it stop closing when something is missing?
p50 / p95 latency per speech line at 1, 4, 8 rooms	scale.sh + collect.py	An alert must arrive before the surgeon closes; shows how far one Nano scales
Memory used at 8 rooms	free -h during the 8-room run	Shows we used the GB10's 128 GB for real work, not a toy
Cloud escalation rate and bytes sent	escalation_eval.py + ledger	Makes the edge-vs-cloud decision measurable, as the challenge requires
Patient data sent to cloud	Presidio double-check before every cloud call; ledger bytes are anonymous text only	Proves the data-residency claim

12. Chaos mode and the demo

12.1 Cut the cloud (live, safe)

Click **Cut cloud link** on the dashboard (it creates the file `state/OFFLINE`). Ask a question: the route shows 'local (cloud offline)'. Click **Reconnect cloud** to undo. Terminal equivalent: `touch state/OFFLINE` and `rm state/OFFLINE`.

WATCH OUT: Do NOT unplug the Nano's network cable live. Your laptop reaches the dashboard through that network, so the screen would freeze. Do the real cable pull only in the recorded video, filmed on a monitor plugged into the Nano.

12.2 Kill the vision model (live)

```
tmux kill-session -t vlm          # tiles switch to "AUDIO ONLY - camera agent down"
./start.sh                       # restarts it; the 27B session reports it is already running
```

12.3 Tamper with the record (live)

```
nano state/ledger.jsonl          # open the log in a simple text editor
# change any number on line 3, press Ctrl+O, Enter to save, Ctrl+X to exit
# the dashboard shows "RECORD TAMPERED at entry #3" within a second
```

To reset afterwards: `rm state/ledger.jsonl` (a new chain starts automatically).

12.4 Five-minute demo script

Time	Say / do	Screen
0:00	Hook: 'Surgical never events, like a sponge left inside a patient, still happen. Meet the circulating nurse.'	Title slide or mock OR photo
0:30	Start 8 rooms: <code>python app/simulate.py --ors 8 --speed 3 --problem 7</code>	8 tiles appear
1:00	Open room 2's live transcript; point at the Counter and Checklist working	Transcript expander
1:30	Room 7: 'Audio says 10 in, 10 out. Camera sees 9. Closing is BLOCKED.'	Red tile + Critic sentence
2:15	Privacy split-screen: ask the prefilled question	Left: full record. Right: anonymous text or 'nothing left'
2:45	Cut cloud link; kill the vision model; system keeps working	OFFLINE banner; AUDIO ONLY tiles
3:30	Evidence: scale chart and base vs fine-tuned chart	scale.png, vision.png
4:15	'Why this only works on a ZGX Nano': 8 rooms, two models, training and serving on one box	Architecture slide
4:45	Close: 'One ZGX Nano per surgical floor. OR Sentinel.'	Final slide

12.5 Answers to the questions judges will ask

Question	Answer
Why not RFID-tagged sponges?	RFID adds a cost to every sponge and needs special readers. We use cameras many ORs already have and also cover untagged instruments.
Why does this need a GB10?	One room fits on a laptop. One GB10 runs 8 rooms with a 27B model and a vision model together. Here is the latency chart.
Where is the cloud?	Urgent or patient-identifying requests always stay local. Only anonymous, non-urgent protocol questions go out, after a double check. We measure the escalation rate.
Nurses don't always say counts aloud.	That is why the camera exists. If audio and vision disagree, it flags it.

Question	Answer
How is this different from the existing OR Black Box product?	That product mostly analyzes recordings after surgery. Ours works live, inside the hospital, before closing.
What if the AI is wrong?	Rules decide the block, not the AI. The nurse's count stays official; we never auto-silence an alert.
Is the data real?	No. Acted audio and photos from a mock OR we built. No real patient data.

13. Package for replicability

13.1 .gitignore and Dockerfile

File: .gitignore

```
.venv/  
.env  
state/  
__pycache__/  
*.log  
*.tmp
```

File: Dockerfile

```
# Container for the OR Sentinel app. The AI models still run on the Nano via ZRT;  
# --network host lets the container reach them on ports 8000 and 8001.  
FROM python:3.11-slim  
WORKDIR /app  
COPY requirements.txt .  
RUN pip install --no-cache-dir -r requirements.txt \  
    && python -m spacy download en_core_web_lg  
COPY . .  
EXPOSE 8501  
CMD ["streamlit", "run", "app/dashboard.py", "--server.address", "0.0.0.0", "--server.port", "8501"]  
  
cd ~/or-sentinel  
docker build -t or-sentinel . # build the container image (several minutes)  
docker run --network host or-sentinel # run the dashboard; --network host lets it reach ZRT on 8000/8001
```

13.2 README.md template (copy, then fill in the brackets)

```
# OR Sentinel - live surgical safety AI on one HP ZGX Nano  
  
**User:** circulating nurse. **Problem:** retained surgical sponges ("never events").  
**Solution:** one ZGX Nano hears and sees the sponge count in 8 operating rooms at once  
and blocks closing if anything is missing. Patient data never leaves the hospital.  
  
## Architecture  
[paste the diagram from Part 1 or a PNG]  
  
## Edge vs cloud decision  
Local first. Cloud only if: not urgent AND local confidence < 0.7 AND cloud reachable  
AND Presidio finds no patient data after stripping. Every decision is logged.  
  
## How to run  
1. `sudo snap install --classic zrt`  
2. `python3 -m venv .venv && source .venv/bin/activate && pip install -r requirements.txt`  
3. `python -m spacy download en_core_web_lg`  
4. Edit model names in `start.sh` and `app/config.py`, then `./start.sh`  
5. `python app/simulate.py --ors 8 --speed 3 --problem 7`  
6. `streamlit run app/dashboard.py --server.address 0.0.0.0 --server.port 8501`  
  
## Models  
- Reasoning: Inference/Qwen3.8-27B-NVFP4 via ZRT  
- Vision: [your HF model link], fine-tuned on the Nano from Qwen2.5-VL-7B-Instruct  
  using HP's Med_VLM_Fine-Tune_vLLM pipeline on [N] photos from our mock OR  
  
## Metrics (how and why)  
| Metric | Result | Why |  
| Sponge count accuracy base -> fine-tuned | [x]% -> [y]% | proves on-device training |  
| p95 latency at 1 / 4 / 8 rooms | [a] / [b] / [c] s | alert must beat the surgeon closing |  
| Memory at 8 rooms | [m] GB of 128 GB | shows real use of the GB10 |  
| Cloud escalation rate | [e]% of 20 questions | makes the edge/cloud decision measurable |  
| Patient data sent to cloud | 0 | data residency |  
| Mismatch scenarios caught | [k] of [n] | the safety claim |  
  
## What we learned pushing the Nano  
[e.g. "Serving the 27B and vision models together at 0.50/0.25 memory handled 8 rooms  
at p95 [c] s; training while serving roughly doubled latency, so we trained overnight."]  
  
## Limits  
Synthetic data: acted audio and a mock OR. Not a medical device. The nurse's count is official.
```

13.3 Push to a public GitHub repo

1. On github.com, click **New repository**, name it or-sentinel, choose **Public**, do NOT add a README (you have one). Click Create.
2. GitHub Settings, Developer settings, Personal access tokens, **Tokens (classic)**, Generate new token, tick **repo**. Copy it. Git asks for this instead of a password.
3. Run:

```
cd ~/or-sentinel
git config --global user.name "<Your Name>"
git config --global user.email "<you@sjsu.edu>"
git init                # start tracking this folder
git add .               # stage every file (except those in .gitignore)
git status              # check: no .venv, no state/, no tokens listed
git commit -m "OR Sentinel: edge AI surgical safety on HP ZGX Nano"
git branch -M main
git remote add origin https://github.com/<your-github-username>/or-sentinel.git
git push -u origin main # username = GitHub username, password = the token
# every few hours afterwards:
git add . && git commit -m "progress" && git push
```

WATCH OUT: The Nano will be wiped at the end of the event. Anything not pushed to GitHub (and your model on Hugging Face) is lost. Also commit bench/results/ with your CSVs and charts.

14. Record, submit and post

14.1 Two-minute video (YouTube, Public)

Seconds	Content
0-15	Problem and user: never events; the circulating nurse
15-45	Dashboard: 8 rooms, the hidden-problem room turning red
45-70	Privacy split-screen and the edge vs cloud rule
70-90	Chaos: real cable pull filmed on the Nano's own monitor; vision model killed
90-110	Charts: scale and fine-tuning gain
110-120	'One ZGX Nano per surgical floor.' Team names, repo link

Record the screen with OBS Studio (free) or your laptop's built-in recorder (Mac: Cmd+Shift+5; Windows: Win+Alt+R with Xbox Game Bar). Upload to YouTube, set visibility to **Public**, copy the URL. Also record a full 5-minute demo as a backup in case the live demo fails.

14.2 Final checklist

- [] Public GitHub repo: code, README.md, Dockerfile, start.sh, requirements.txt
- [] Fine-tuned model public on Hugging Face and linked in the README
- [] bench/results/ committed: scale.csv, vision_*.json, escalation.txt, scale.png, vision.png
- [] Metrics with how and why (README table)
- [] 5-minute demo: live over SSH or tunneled port, plus the recorded backup
- [] 2-minute video on YouTube set to Public; URL submitted along with the video file
- [] Interactive presentation: the live dashboard plus slides with architecture, benchmarks and impact visuals
- [] SJSU Google Drive: project brief, pitch video, links, presentation, diagrams, mock-OR photos
- [] Social posts tagging @SJSUAppliedDataScience @SJSUAIInstitute @HackSJSU @HPInc @ZbyHP @NVIDIA @Arm @Canonical @Salesforce @iTradeNetwork @EdgeAI @LocalAgentic
- [] Submitted by 7pm (deadline 8pm)

15. Troubleshooting

15.1 What might break and the fallback

Part	Confidence	What might break	Fallback
ZRT install and serving	High	Model name format (with or without hf:)	Try both forms; copy the id from /v1/models
Two models at once	High	Out of memory	Lower --gpu-memory-utilization and --max-model-len
Adapting HP's fine-tune script	Medium	Data format or field names differ	Part 6.2 converter or HF dataset upload; base model after Fri 10am
Training time	Medium	Runs past morning	Fewer photos or epochs; start by Thu 11pm
faster-whisper on the Nano	Medium	Install or GPU issues on ARM	CPU mode (default) or transcribe on a laptop
27B JSON replies	Medium	Extra text or thinking around the JSON	Handled by _extract_json; add ' /no_think' if slow
Vision accuracy	Medium	Overlapping pads miscounted	Report 'within 1' too; add clearer photos
Presidio	Medium	Misses an unusual name	Double-check keeps it local; test your demo sentence
Live cable pull	Low	Cuts your own dashboard connection	Dashboard switch live; real pull only on video
Nano restart	Medium	Tailscale goes down	Lab keyboard: sudo tailscale up, then ./start.sh
Node wiped at end	Certain	Unpushed work lost	git push every few hours

15.2 Error messages

Message contains	Meaning	Fix
ModuleNotFoundError	venv not active, or package missing	source ~/or-sentinel/.venv/bin/activate; pip install -r requirements.txt
Connection refused (port 8000/8001)	Model not running or still loading	tmux attach -t llm or tail -f llm.log; wait; ./start.sh
404 / model not found	Wrong model name in config.py	Copy the id from curl localhost:8000/v1/models
CUDA out of memory	Too much running at once	./stop.sh, lower memory flags, or wait for training
permission denied ... docker	zrt not in docker group	sudo usermod -aG docker zrt, then a supervised restart
Address already in use	Two programs on one port	./stop.sh; check tmux ls; use different ports
FlashInfer ...	Library version mismatch	Paste the full error into an AI assistant; check the vLLM Recipes page
No transcripts in data/audio/	transcribe.py not run	Part 8
No photos found	Photos not in data/photos or wrong extension	Part 4.2; use .jpg, .jpeg or .png
All tiles stuck on 'Language model unavailable'	27B service down or too slow	Check port 8000; lower rooms or speed

Message contains	Meaning	Fix
Dashboard blank / not reachable	Port not forwarded	VS Code Ports tab: forward 8501; or use the Tailscale IP
spacy / en_core_web_lg not found	Presidio's language model missing	<code>python -m spacy download en_core_web_lg</code>

When stuck for more than 20 minutes on anything, switch to the fallback and move on. A complete, honest system beats a perfect half-system.